

NAMOSIM: a Robot Motion Planner for Navigation Among Movable Obstacles

David Brown^{1,4}, Jacques Saraydaryan^{1,3,4}, Benoit Renault^{1,2,4}, and Olivier Simonin^{1,2,4}

1 Inria, CHROMA Team 2 INSA Lyon 3 CPE Lyon 4 CITI Laboratory

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

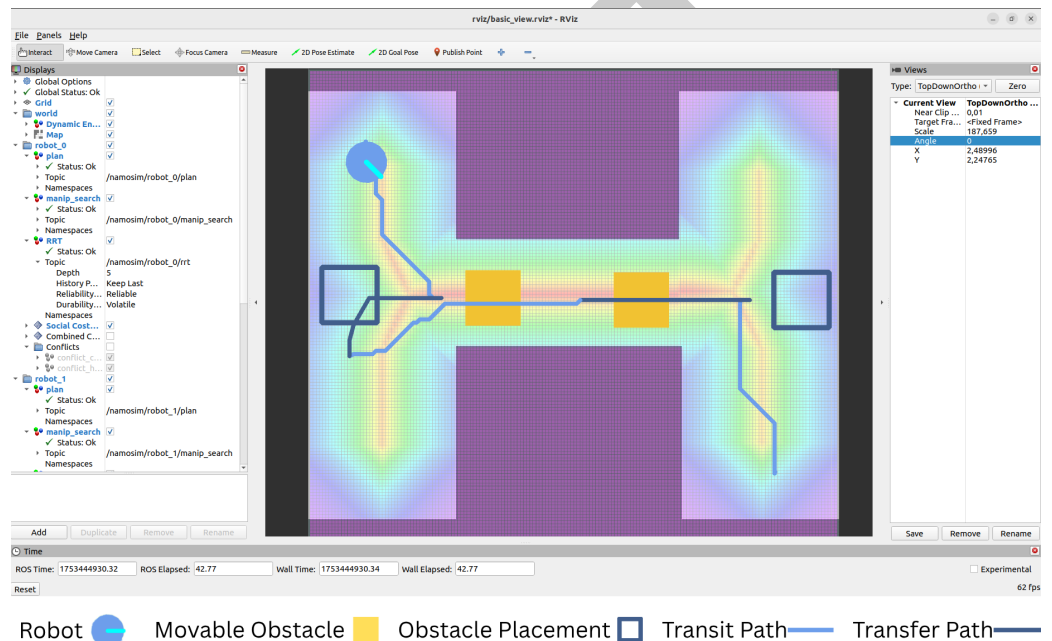


Figure 1: A NAMOSIM scenario with one robot and two obstacles is visualized in RViz. The robot's plan is shown by the blue lines. The darker shade of blue indicates the part of the path involving an obstacle transfer. The empty rectangles indicate the planned obstacle placements. The social costmap is seen in the rainbow background.

Summary

NAMOSIM is a mobile robot motion planner and simulator designed for the problem of Navigation Among Movable Obstacles (NAMO). The planner simulates robots navigating in 2D polygonal environments where certain obstacles can be grasped and relocated to enable robots to reach their goals. NAMOSIM extends the classic navigation problem with a layer of interactivity, posing interesting research questions while remaining well-defined and amenable to various algorithmic approaches. NAMOSIM is intended for researchers and developers working on robot navigation in dynamic environments, particularly where physical interaction is necessary.

NAMOSIM supports the development of custom NAMO algorithms using a modular agent-based architecture. It includes a baseline agent implementing Stilman's NAMO algorithm (Stilman & Kuffner, 2005) and incorporating a communication-free coordination strategy for multi-robot scenarios (Renault et al., 2024). A variety of other agent behaviors are implemented, and new

agents utilizing alternative approaches can be integrated into the planner by implementing the **Agent** base class. NAMOSIM thus supports reproducible research in single and multi-robot NAMO algorithms.

NAMOSIM is packaged as a ROS 2 (Steven Macenski et al., 2022) package to facilitate integration with commonly-used robotics tools such as Nav2 (Steve Macenski et al., 2020) and Gazebo (Koenig & Howard, 2004), but it can also be used as a standalone Python module. Simulations are displayed in a Tkinter window and ROS 2 messages are published for more-detailed visualization in RViz (Kam et al., 2015). Several prebuilt scenarios for testing and benchmarking are included. These are stored as SVG files, allowing for convenient creation of custom scenarios using a free SVG editor such as Inkscape.

Statement of Need

Many interesting applications in autonomous mobile robotics involve physical interaction with the environment as well as social coordination with other agents. However, global navigation planners typically assume static environments, leaving complex behaviors to be managed by separate software components, complicating implementation. Ideally, motion planners should be able to reason about physical and social interactions and adapt to changing conditions. NAMOSIM addresses this challenge by providing an open-source planner and simulation environment designed for single and multi-robot NAMO algorithms, which require not only navigation but also reasoning about which obstacles to move, where to move them, and how to coordinate with other agents.

While there are several prior works on NAMO (Leviñh et al., 2013, 2014; Scholz et al., 2016; Wu et al., 2010; Zhang et al., 2023), they lack multi-robot capability and do not provide reusable open-source tools designed for real-world deployment on physical robots. There is also a large body of work on multi-agent path finding (MAPF), but this does not consider movable obstacles. Another innovative related work (Baker et al., 2020) involved both multiple agents and movable obstacles, but was not designed for navigation nor for robotics applications. NAMOSIM fills this gap by providing a versatile software package for experimenting with single and multi-robot NAMO algorithms, supporting the robotics community to create more capable and adaptive robotic systems.

Major Features

NAMOSIM provides a robust set of features to support research and development in Navigation Among Movable Obstacles (NAMO):

- **Modular Agent-Based Architecture:** The simulator is built around a flexible Agent interface, allowing users to implement and test custom NAMO planning algorithms. A baseline NAMO algorithm implementation is available for immediate use and benchmarking.
- **Support for Multiple Robot Models:** NAMOSIM supports both holonomic and differential-drive robot models, enabling realistic simulation of various robotic platforms.
- **ROS 2 Integration:** NAMOSIM forms a ROS 2 package, enabling seamless integration into simulated and physical robotics projects and visualization via RViz.
- **2D Environment Simulation:** The simulator provides a customizable 2D environment where users can define static and movable obstacles, supporting complex scenarios for testing multi-robot coordination strategies and NAMO algorithms.
- **Prebuilt Scenarios and Tests:** NAMOSIM includes several custom scenario files for benchmarking and testing specific situations.
- **Multi-Robot Coordination:** The simulator supports multi-robot scenarios, and our baseline agent implements a communication-free coordination strategy (Renault et al., 2024).

66 These features make NAMOSIM a versatile tool for prototyping, evaluating, and deploying
67 NAMO algorithms in diverse robotic applications.

68 Customizable Scenarios

69 NAMOSIM environments, or **scenarios**, are stored in SVG format and can be edited using any
70 SVG editor, such as Inkscape. The scenario SVG file contains the following key elements:

- 71 ■ The geometry of the static map
- 72 ■ The polygons and orientations of all robots and movable obstacles
- 73 ■ Configuration settings that define the behavior of the environment and robots

74 The static map can also be included as an image layer within the SVG to conveniently
75 incorporate ROS grid-map images generated by standard mapping tools.

76 Architecture

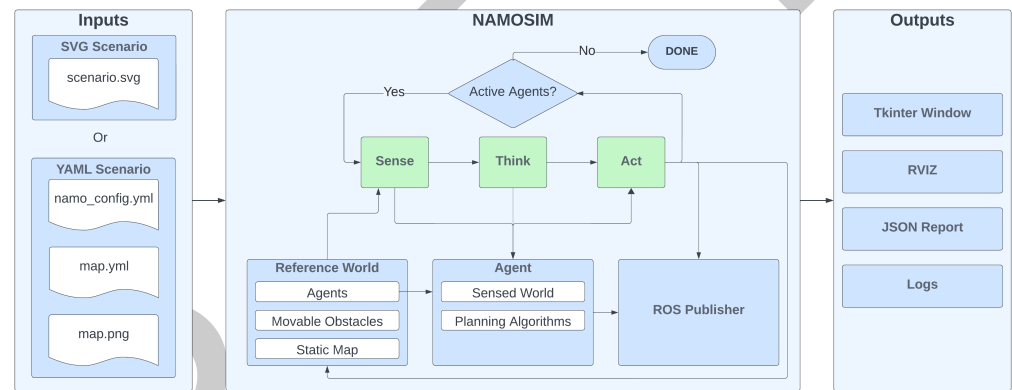


Figure 2: NAMOSIM High-Level Architecture

77 At a high level, NAMOSIM executes a SENSE-THINK-ACT loop that performs the following
78 functions at each iteration:

- 79 1. **SENSE**: Each agent senses the environment and updates its internal representation.
- 80 2. **THINK**: Each agent computes a new plan or updates its current plan.
- 81 3. **ACT**: Each agent selects a single discrete action to execute.

82 The loop is expected to execute at a regular frequency, with the assumption that all agent
83 functions run sequentially in a synchronized manner.

84 Stilman's NAMO Algorithm

85 NAMOSIM includes a baseline implementation of Stilman's 2005 NAMO algorithm (Stilman
86 & Kuffner, 2005). The key idea of this algorithm is to move obstacles to merge disjoint
87 components of the robot's free configuration space. The map is divided into a set of disjoint
88 **connected components**, where each grid cell in a given component is reachable from all other
89 cells in the same component. It can be proven that components are separated by movable
90 obstacles or are otherwise unreachable. The algorithm functions by moving obstacles to join
91 components until the robot's current component includes the goal cell.

92 The algorithm works by recursively performing the following two stages:

- 93 1. **SELECT_OBSTACLE_AND_COMPONENT**: The first stage performs a simplified
94 A* grid search, allowing the agent to pass through movable obstacles. It returns the ID
95 of the first movable obstacle encountered on the optimal path to the goal and the ID of
96 the component encountered after passing through the obstacle.
- 97 2. **OBSTACLE_MANIPULATION_SEARCH**: The second stage finds a **transit path**
98 from the robot's current position to a grasp pose near the obstacle. Then, it finds a
99 **transfer path** by performing an obstacle manipulation search to join the robot's current
100 component to the component selected in stage 1. If this stage fails, the obstacle and
101 component pair are added to an avoid-list, and the algorithm returns to stage 1.

102 Each iteration of the algorithm continues with a copy of the environment where the robot
103 and obstacle start from the poses resulting from the previous obstacle manipulation search.
104 See also (Renault, 2023) for more details. (Wu et al., 2010) extended Stilman's algorithm to
105 unknown environments where obstacle movability is ascertained through interaction. We hope
106 to implement this idea in NAMOSIM in future work.

107 Collision Detection

108 Custom agents are free to implement their own collision detection routines; however, the
109 baseline agent detects collisions using a simple binary-occupancy grid during transit paths
110 (when not carrying an obstacle), assuming a circular robot footprint. When transporting a
111 movable obstacle, the robot footprint is non-circular, and collision detection is based on the
112 **convex swept volume** resulting from the area swept by the combined robot-obstacle footprint
113 due to the action motion (Jiménez et al., 1998). Although computationally expensive, this
114 ensures all possible collisions are detected, regardless of the shape of the robot or obstacle.

115 Social Costmap

116 A novel contribution in the baseline implementation is the option to use a social costmap
117 during the obstacle manipulation search to guide obstacle placement decisions. This allows
118 robots to place obstacles in areas less likely to block the free passage of other agents, including
119 humans, reducing the likelihood that obstacles will need to be moved again. The key heuristic
120 of the social costmap is to **avoid narrow corridors and central areas**, assigning higher costs to
121 narrow corridors and the centers of open spaces. This helps robots avoid placing obstacles in
122 front of doorways or in the center of rooms. The social costmap is explained in greater detail
123 in (Renault et al., 2020; Renault, 2023).

124 Conflict Avoidance and Deadlock Resolution

125 NAMOSIM's baseline agent can avoid conflicts and resolve deadlocks with other agents.
126 Conflict avoidance works by looking ahead along the agent's current plan for a fixed number
127 of steps, called the **conflict horizon**. Within this horizon, the agent simulates each planned
128 action and checks for potential conflicts. For example, the agent may have planned to move
129 an obstacle that is no longer at the expected location, or another robot may be crossing the
130 planned path within the conflict horizon, raising the potential for a collision.

131 The baseline agent avoids conflicts by either pausing or replanning around them. A **deadlock** is
132 detected when the same conflict configuration is repeatedly encountered, even after replanning.
133 To resolve deadlocks, the agent follows an evasion strategy which is optionally based on the
134 local social costmap (Renault et al., 2024).

135 Acknowledgements

136 This research was supported by the Inria NAMOEX initiative.

137

- 137

183 Zhang, K., Lucet, E., Sandretto, J. A. dit, & Filliat, D. (2023). Navigation among movable
184 obstacles using machine learning based total time cost optimization. *2023 IEEE/RSJ*
185 *International Conference on Intelligent Robots and Systems (IROS)*, 11321–11327. <https://doi.org/10.1109/IROS55552.2023.10341355>
186

DRAFT